

Mini-Projet Android

CityPulse — Explorez votre ville en temps réel

Module	Introduction à l'architecture Android
Niveau	Débutant / Intermédiaire
Travail	En groupe (max 4 étudiants)
Durée	4 semaines
Livrable	APK + code source + rapport PDF (5 pages max)

1. Contexte et objectifs

Vous venez de découvrir l'architecture d'Android, la gestion des permissions, les bases du développement natif (géolocalisation, partage, etc.) et le déploiement sur le Google Play Store. Il est temps de mettre tout cela en pratique dans un projet concret qui vous poussera également à explorer des notions avancées.

CityPulse est une application mobile Android permettant à un utilisateur de découvrir les lieux d'intérêt autour de lui, de les sauvegarder en favoris, de laisser des notes personnelles, et de partager ses découvertes avec ses contacts. L'application doit fonctionner en mode connecté **et** en mode hors-ligne.

Objectifs pédagogiques :

- Comprendre et exploiter les couches de l'architecture Android (Linux Kernel, HAL, ART, Framework, Applications).
- Gérer correctement le cycle de vie des composants (Activity, Fragment, Service).
- Implémenter les permissions à l'exécution (runtime permissions) selon les bonnes pratiques.
- Utiliser les API de géolocalisation (Fused Location Provider).
- Persister des données localement avec Room (SQLite).
- Communiquer avec une API REST externe (Retrofit / OkHttp).
- Appliquer un patron d'architecture (MVVM recommandé).
- Partager du contenu via les Intents implicites.
- Préparer le déploiement (signature APK, configuration du manifest, store listing).

2. Fonctionnalités attendues

2.1 Écran Carte interactive

Afficher une carte (Google Maps SDK ou OpenStreetMap via osmdroid) centrée sur la position actuelle de l'utilisateur. Les lieux d'intérêt proches doivent apparaître sous forme de marqueurs. Un clic sur un marqueur affiche une infobulle avec le nom, la catégorie et la distance estimée.

- Demander la permission `ACCESS_FINE_LOCATION` à l'exécution.

- Gérer le cas où l'utilisateur refuse la permission (message explicatif + fonctionnement dégradé).
- Mettre à jour la position en arrière-plan via un `ForegroundService` avec notification persistante.

2.2 Liste et recherche de lieux

Proposer une vue liste (`RecyclerView`) des lieux à proximité, avec filtrage par catégorie (restaurants, parcs, musées, commerces...) et recherche textuelle. Les données sont récupérées depuis une API REST publique (ex. : Overpass / OpenTripMap / Foursquare / Google Places). Si aucune de ces solutions ne vous convient, vous pouvez créer votre propre base de données en utilisant [SupaBase](#).

- Utiliser `Retrofit` + `Gson` / `Moshi` pour les appels réseau.
- Implémenter un mécanisme de cache (base Room) pour le mode hors-ligne.
- Afficher un indicateur de chargement et gérer les erreurs réseau.

2.3 Fiche détail d'un lieu

Chaque lieu dispose d'un écran de détail affichant : nom, adresse, photo (si disponible), catégorie, coordonnées GPS, et un champ de notes personnelles modifiable par l'utilisateur. L'utilisateur peut ajouter le lieu à ses favoris.

2.4 Favoris et persistance locale

Les favoris sont stockés dans une base de données locale via **Room**. L'écran Favoris affiche la liste des lieux sauvegardés, accessible même sans connexion internet. L'utilisateur peut supprimer un favori par un swipe ou un appui long.

2.5 Partage

Depuis la fiche détail, l'utilisateur peut partager un lieu via un **Intent implicite** (SMS, e-mail, WhatsApp, etc.). Le message partagé doit contenir le nom du lieu, ses coordonnées et un lien vers Google Maps.

2.6 Notifications

Lorsqu'un nouveau lieu d'intérêt est détecté à moins de 500 m de l'utilisateur, une notification locale est envoyée. Gérer le canal de notification (Notification Channel) et la permission `POST_NOTIFICATIONS` (Android 13+).

3. Contraintes techniques

Contrainte	Détail
Langage	Kotlin (Java accepté mais Kotlin recommandé)
SDK minimum	API 26 (Android 8.0)
Architecture	MVVM (ViewModel + LiveData / StateFlow)
Réseau	Retrofit 2 + OkHttp
Base locale	Room (SQLite)
Géolocalisation	Fused Location Provider (Google Play Services)
Carte	Google Maps SDK ou osmdroid
UI	Material Design 3 (Material You)
Versionnement	Git (dépôt GitHub / GitLab obligatoire)

3.1 Architecture attendue (MVVM)

Votre projet doit respecter une séparation claire des responsabilités. Voici la structure recommandée :

- **View (Activity / Fragment)** : affichage, interactions utilisateur. Ne contient aucune logique métier.
- **ViewModel** : expose les données sous forme de LiveData ou StateFlow. Orchestre les appels au Repository.
- **Repository** : source unique de vérité. Décide s'il faut interroger l'API distante ou la base locale.
- **Data Sources** : Room DAO (local) et Retrofit Service (distant).

Conseil : créez un package par couche (ui/, viewmodel/, repository/, data/local/, data/remote/) pour garder votre code organisé.

4. Fonctionnalités bonus (points supplémentaires)

Ces fonctionnalités ne sont pas obligatoires mais permettent d'obtenir des points bonus et démontrent une maîtrise approfondie :

- ★ **Mode sombre (Dark Theme)** : supporter le thème clair et sombre avec basculement automatique selon les préférences système.
- ★ **Widget Home Screen** : afficher les 3 derniers favoris directement sur l'écran d'accueil.
- ★ **Geofencing** : déclencher une alerte lorsque l'utilisateur entre dans un périmètre prédéfini autour d'un lieu favori.
- ★ **Tests unitaires** : écrire des tests pour le ViewModel et le Repository (JUnit + Mockito).
- ★ **CI/CD** : configurer un pipeline GitHub Actions pour builder et tester automatiquement à chaque push.
- ★ **Accessibilité** : rendre l'application compatible avec TalkBack (contentDescription, focus order).
- ★ **Internationalisation** : supporter au moins 2 langues (français + anglais) via les fichiers strings.xml.

5. Consignes importantes

- **Plagiat** : tout code copié sans attribution sera sanctionné. Vous pouvez utiliser des tutoriels et de la documentation, mais citez vos sources dans le rapport.
- **Git** : un historique de commits régulier et significatif est attendu. Tous les membres du groupe doivent contribuer de manière équilibrée.
- **Clé API** : ne publiez jamais vos clés d'API dans le dépôt Git. Utilisez un fichier `local.properties` ou des variables d'environnement.
- **Présentation orale** : chaque groupe dispose de 10 minutes (5 min de démo + 5 min de questions). Préparez un scénario de démonstration réaliste.
- **Rapport** : le rapport doit inclure l'architecture choisie, les difficultés rencontrées, les captures d'écran, et une conclusion sur les apprentissages.
- **Utilisation des Agents IA** : Si vous utilisez des agents IA pour vous accompagner dans vos tâches, mentionnez-le dans le document de rapport. Inclure également les instructions fournies à l'agent dans un fichier [AGENTS.md](#).

6. Modalités de rendu

Un répertoire partagé vous sera communiqué ultérieurement. Vous y déposerez un fichier compressé nommé comme suit PRENOM-NOM_PRENOM-NOM_.... et dont le contenu est composé comme suit :

- Le rapport de projet
- Les sources de l'application
- Un fichier README